

LangurTrace: Forensic analysis of local LLM applications

LangurTrace: 로컬 LLM 애플리케이션에 대한 포렌식 분석

저자: Sungjo Jeong, Sangjin Lee, Jungheum Park

학회/저널: *Forensic Science International: Digital Investigation*, Volume 54,
Supplement, October 2025, 301987

1. 연구 배경

- LLM의 급격한 확산과 로컬 배포 증가
 - DeepSeek, Llama 등 경량화된 모델과 Hugging Face 같은 플랫폼의 발달로 개인 PC에서 직접 실행할 수 있는 LLM 애플리케이션의 사용이 크게 증가
- 사이버 범죄 및 불법 행위 악용
 - LLM이 피싱 이메일 작성, 악성코드 생성, CSAM 제작 등 범죄 도구로 악용되는 사례 증가
- 포렌식 연구의 부재
 - 기존 디지털 포렌식 분야에서는 LLM을 수사 보조 도구로 활용하거나 SaaS 서비스 분석에만 집중, 로컬 LLM 애플리케이션이 남기는 디지털 아티팩트에 대한 체계적인 포렌식 연구는 미진

2. 연구 목적

- 구조화된 환경 프레임워크 제안
 - 다양한 로컬 LLM 애플리케이션을 구성 요소로 분류하는 일반화된 시스템 모델을 구축
- 아티팩트 식별 및 분류
 - 대표적인 로컬 LLM 애플리케이션(Ollama, LM Studio..)들을 분석하여 프롬프트, 채팅 기록, 업로드/생성 파일, 모델 설정 기록 등 수사에 필요한 주요 아티팩트를 식별하고 유형화
- 자동화 포렌식 도구 개발 및 공개
 - 식별된 아티팩트를 자동으로 수집, 파싱하여 실무에서 활용할 수 있는 오픈소스 도구인 **LangurTrace**를 개발하고 검증

3. Local LLM 환경 구조

- 로컬 LLM 시스템 구조를 크게 3가지 주요 구성 요소와 클라우드 연동 요소로 분류

1) 백엔드 런타임

- 모델 딥러닝 레이어 실행, 모델 다운로드 및 관리를 담당 (예: Ollama)
- 모델 허브(Hugging Face 등)에서 GGUF 등의 포맷을 받아 실행하며 API 엔드포인트를 제공

2) 클라이언트 인터페이스

- 사용자가 대화할 수 있는 GUI를 제공 (예: Chatbox)
- 백엔드 API와 통신하며, 대화 세션 설정, 대화 기록, 사용자 지정 프롬프트, 업로드/생성 파일 등을 관리

3) 통합 플랫폼

- 백엔드 런타임과 클라이언트 인터페이스 기능을 단일 앱으로 combined한 형태 (예: LM Studio, Msty, Jan, GPT4All)

4) 클라우드 연동

- 로컬 인터페이스에서 API 키를 통해 클라우드 모델을 호출하는 방식
- 로컬에는 주로 상태 비저장(Stateless) API 요청 정보와 클라이언트 측 대화 기록만 저장

4. 실험 방법

- 실험 환경: Windows 11 Pro (24H2) 환경
- 분석 대상 애플리케이션:

백엔드: Ollama

클라이언트: Chatbox

통합 플랫폼: LM Studio, Msty, Jan, GPT4All

클라우드 연동: ChatGPT (GPT-3.5/4o, DALL-E 3)

- 데이터셋 생성 및 시나리오:

- 1) 모델 다운로드 및 삭제 (백엔드)
- 2) 클라우드 API 키 등록 및 삭제 (클라이언트)
- 3) 대화 세션 생성 및 삭제 (클라이언트)
- 4) 텍스트 대화, 파일 업로드(PDF, 이미지 등), 파일 생성 수행

실행 순서에 따른 편향을 줄이기 위해 각 작업을 애플리케이션당 최소 50회 이상 반복 실행한 후 디스크 변경 사항(생성·수정·삭제)을 모니터링

- Ollama에서 생성 및 관리되는 아티팩트

Artifact	Format	설
서버 로그	텍스트	모델 설정, API 호출
모델 매니페스트	도커 매니페스트	모델 레이어의 메타데이터
모델 레이어	이진수	템플릿, 매개변수
앱 로그	텍스트	애플리케이션 사용 로그
업그레이드 로그	텍스트	소프트웨어 업데이트 내역
CLI 기록	텍스트	CLI를 통한 사용자 입력

- Chatbox에서 생성 및 관리되는 아티팩트

Artifact	Format	설
config.json	JSON	채팅 세션, API 키
업로드된 파일	Base64로 인코딩됨	사용자가 업로드한 파일
생성된 파일	Base64로 인코딩됨	LLM에서 생성된 파일
주요 로그	텍스트	백업 및 업데이트 내역
API 캐시	크롬 캐시	API 호출에 대한 응답
모델 목록	레벨DB	공급업체별 모델 목록

- 통합 플랫폼 애플리케이션의 산출물

★: 사용자가 삭제한 후에도 아티팩트가 존재하며 복구될 수 있습니다.

☆: 삭제된 파일은 존재하지만 일반적으로 사용자가 삭제한 후에는 복구할 수 없습니다.

✕: 찾을 수 없음

- : 지원되지 않는 기능

	LM Studio	Msty	Jan	Gpt4All
다운로드된 모델	★	★	★	☆
모델 설정 내역	☆	☆	☆	✕
채팅 설정	☆	★	★	☆
채팅 기록	☆	☆	★	☆
업로드된 파일	☆	★	-	☆

	LM Studio	Msty	Jan	Gpt4All
생성된 파일	-	-	-	-
API 키	-	☆	★	☆

로컬 LLM 애플리케이션과 관련된 아티팩트의 전체 경로와 목록을 확인하려면 **부록 A, 부록 B** 참조

5. LangurTrace 도구 구조 및 구현

- LangurTrace: Windows 환경의 로컬 LLM 아티팩트 수집 및 파싱을 자동화하는 오픈 소스 포렌식 도구

- 주요 모듈 구조:

- 1) Collector : 시스템 내 대상 애플리케이션의 아티팩트 파일(JSON, SQLite, DB, 로그 파일 등)을 스캔하고 출력 디렉터리로 복사
- 2) Parser : 수집된 아티팩트 중 의미 있는 데이터(대화 기록, 모델 다운로드 URL, 삭제된 흔적, API 키 등)만 선택적으로 추출
- 3) Reporter : 파싱된 결과를 조사관이 한눈에 파악할 수 있도록 HTML 및 CSV/XLSX 형식으로 변환

6. 실험 결과 및 평가

- 삭제 후 복원력 평가 (UI 레벨 삭제 기준):

- 1) Ollama, Jan: UI에서 삭제하더라도 로그(`cortex.log` , `server.log` 등)에 다운로드 기록 및 대화 내용이 100% 남아 있어 LangurTrace로 완벽히 복원 가능
- 2) Chatbox, Msty: 삭제된 업로드/생성 파일이 디렉터리에 그대로 남아 있어 Base64 디코딩 등을 통해 100% 복원 가능
- 3) LM Studio, GPT4All: 앱 UI 내 삭제 시 대화 기록 및 파일이 디스크 상에서도 함께 제거되어 기본 파싱 방식으로는 복원률이 낮거나 불가능 (다만 언삭제/슬랙 공간 분석 등 별도 포렌식 기법 적용 여지는 남음)

깃허브 링크: <https://github.com/jeongramon/LangurTrace>

7. 차이점

1) 구조화 DB 비할당 공간(Slack) 및 카빙(Carving) 기법 도입

기존 논문의 한계: 데이터 삭제 시 UI 레벨 삭제 데이터 중 디스크에 잔존하는 파일/로그 파싱에만 의존 (예: GPT4All이나 LM Studio 등 앱 UI에서 완전 삭제된 DB 레코드는 복구 불가로 평가)

차별화 아이디어: SQLite의 WAL(Write-Ahead Log) 파일, FreeList, 또는 LevelDB의 SSTable/WAL 슬랙 공간을 카빙하여 UI에서 삭제되어 DB 레코드상 없어진 채팅 기록이

나 API 키를 복원

2) API 키/자격 증명 기반 2차 클라우드 포렌식 연계 자동화

기존 논문의 한계: 로컬 앱 내부 로그나 DB에서 평문/Base64로 추출된 API 키(OpenAI, Claude 등)를 단순 식별·출력

차별화 아이디어:

API Key 검증 및 세션 추적 자동화: 파싱된 API 키의 유효성을 검증하고, 활성화된 키인 경우 서비스 제공업체(OpenAI API 등)의 Usage Audit Log나 Cloud 기반 대화 세션/Fine-tuning 모델 목록을 API를 통해 자동으로 조회·연동

3) LLM 데이터 특화 포렌식 뷰어 및 LLM-Assisted Triage 도입

기존 논문의 한계: 결과를 HTML 보고서나 Excel(CSV/XLSX) 형태로 정적 파싱하여 수동으로 검토

차별화 아이디어:

타임라인 기반 통합 대화 뷰어: 여러 앱(Ollama + Chatbox + LM Studio 등)을 혼용해 사용한 경우, 각 앱의 대화 로그를 타임스탬프 기준으로 통합하여 하나의 일관된 타임라인(Unified Timeline)으로 보여주는 인터페이스를 제공

자동 요약 및 유해성/의도 분류: 파싱된 대화 수천 건 중 범죄/유해 의도(예: 악성코드 생성 프롬프트, 피싱 문구 작성, 데이터 유출 시도 등)가 의심되는 세션을 키워드 필터링 및 자동 스코어링/요약